
ITHub
Паттерны программирования
Паттерн — это проверенное решение для часто встречающейся проблемы.
• Creational
– Singleton, Factory Method, Builder
• Structural
– Adapter, Decorator, Facade
• Behavioral
– Observer, Strategy, Command
Client
Interface
+ request()
ConcreteA
+ request()
ConcreteB
+ request()
Java
Java

Паттерны программирования (design patterns - шаблоны проектирования) — это проверенные способы решения часто встречающихся задач при разработке программ.

Не изобретать велосипед — использовать известные подходы

Упрощать поддержку — менять одну часть программы с меньшим риском сломать другую

Расширять проект — добавлять новые функции без переписывания всего кода

Работать в команде

SOLID — это пять принципов объектно-ориентированного программирования, которые помогают писать код так, чтобы его было проще поддерживать, тестировать и расширять.

S — Single Responsibility Principle
(принцип единственной ответственности)

O — Open/Closed Principle
(принцип открытости/закрытости)

L — Liskov Substitution Principle
(принцип подстановки Барбары Лисков)

I — Interface Segregation Principle
(принцип разделения интерфейсов)

D — Dependency Inversion Principle
(принцип инверсии зависимостей)

Принцип единственной ответственности

Single Responsibility Principle

Класс должен иметь только одну причину для изменения

Если класс имеет более одной обязанности (когда выполняет слишком много несвязанных задач), то эти обязанности становятся связанными.

Если класс делает слишком много вещей сразу, вам приходится изменять его каждый раз, когда одна из этих вещей изменяется.

При этом есть риск сломать остальные части класса, которые вы даже не планировали трогать.

IThub

IThub

```
class Student {
    void saveToFile() {
        // сохранение данных
    }

void printReport() {
        // формирование отчёта
    }
}
```

```
class Student {
    String name;
}
```

```
class StudentRepository {
    void save(Student student) {
        // сохранение данных
    }
}
```

```
class StudentReport {
    void print(Student student) {
        // формирование отчёта
    }
}
```

Класс Student хранит данные студента, сохраняет их в файл и выводит отчёт.

Если изменится формат отчёта или способ сохранения, придётся менять один и тот же класс.

Принцип разделения интерфейса

Interface Segregation Principle

Клиенты не должны вынужденно зависеть от методов, которыми они пользуются

Если клиенты зависят от методов, которыми они **не** пользуются, то такие клиенты восприимчивы к изменениям в этих методах.

Результатом будет связанность между всеми клиентами.

IThub

```
public interface Payment {

    void initiatePayments();
    Object status();
    Object[] getPayments();
}
```

```
public class BankPayment implements Payment {

    @Override
    public void initiatePayments() {
    }

    @Override
    public Object status() {
        // some logic
    }

    @Override
    public Object[] getPayments() {
        // some logic
    }
}
```

Есть банковское приложение, которое обрабатывает платежи.

На данный момент реализующему классу BankPayment нужны все методы интерфейса Payment.

Таким образом, это не нарушает принцип разделения интерфейса.

IThub

К нам приходит заказчик и говорит, что теперь мы должны обработать платежи по кредиту.

Этот сервис также является разновидностью платежа, но с несколькими дополнительными операциями.

Добавим новые методы в интерфейс Payment и реализуем класс LoanPayment:

```
public interface Payment {

    void initiatePayments();
    Object status();
    Object[] getPayments();

    void initiateLoanSettlement();   // инициировать погашение кредита
    void initiateRePayment();        // инициировать повторный платеж
}
```

IThub

```
public class BankPayment implements Payment {

    @Override
    public void initiatePayments() {
        // some logic
    }

    @Override
    public Object status() {
        // some logic
    }

    @Override
    public Object[] getPayments() {
        // some logic
    }

    @Override
    public void initiateLoanSettlement() {
        // ??
    }

    @Override
    public void initiateRePayment() {
        // ??
    }
}
```

```
public class LoanPayment implements Payment {

    @Override
    public void initiatePayments() {
        //  ??
    }

    @Override
    public Object status() {
        // ??
    }

    @Override
    public Object[] getPayments() {
        //  ??
    }

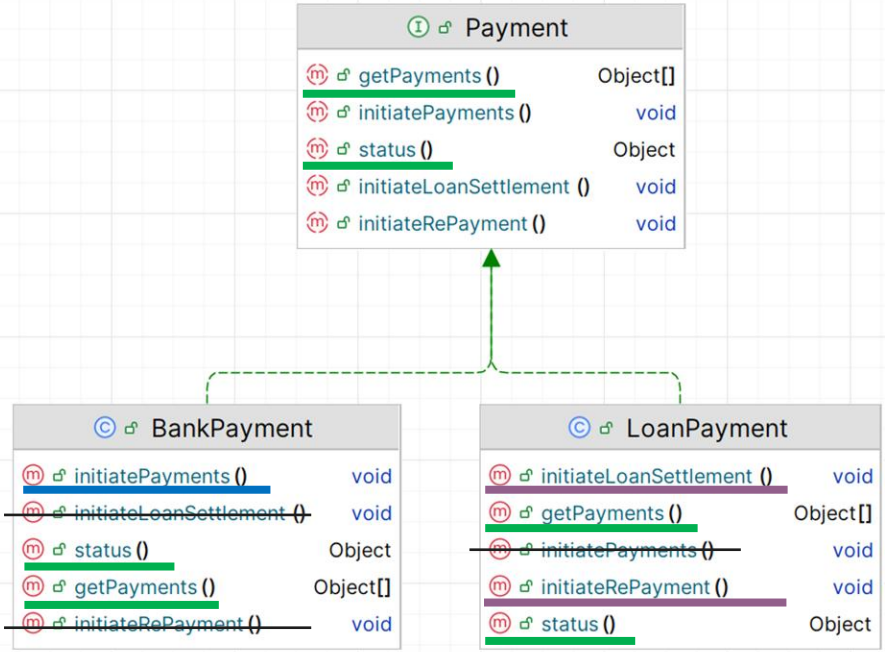
    @Override
    public void initiateLoanSettlement() {
        // some logic
    }

    @Override
    public void initiateRePayment() {
        // some logic
    }
}
```

Теперь, после изменения интерфейса Payment и добавления новых методов, все реализующие классы должны реализовать новые методы.

Проблема в том, что их реализация нежелательна. В данном случае класс реализации LoanPayment должен реализовать метод initialPayments() без реальной необходимости.

Таким образом, принцип нарушается.



- методы **status**() и **getPayments**() обязательны в обеих реализациях
- **initialPayments**() требуется только для BankPayment
- **initialLoanSettlement**() и **initialRePayment**() — только для LoanPayment

Теперь интерфейсы не нарушают принцип.

Реализации не должны предоставлять пустые методы.

Это сохраняет чистоту кода и снижает вероятность ошибок.

```
public interface Payment {

    Object status();
    Object[] getPayments();
}
```

```
public interface iBankPayment extends Payment{

    void initiatePayments();
}
```

```
public interface ILoanPayment extends Payment {

    public void initiateLoanSettlement();

    public void initiateRePayment();
}
```

IThub

```
public class BankPayment implements iBankPayment {

    @Override
    public void initiatePayments() {
        // some logic
    }

    @Override
    public Object status() {
        // some logic
    }

    @Override
    public Object[] getPayments() {
        // some logic
    }
}
```

```
public class LoanPayment implements iLoanPayment {

    @Override
    public Object status() {
        // some logic
    }

    @Override
    public Object[] getPayments() {
        // some logic
    }

    @Override
    public void initiateLoanSettlement() {
        // some logic
    }

    @Override
    public void initiateRePayment() {
        // some logic
    }
}
```

Принцип открытости\закрытости

Open - closed principle

Программные сущности (классы, модули, функции) должны быть открыты для расширения, но закрыты для модификации

1. Они открыты для расширения
   поведение модуля может быть расширено, когда требования приложения изменяются
2. Они закрыты для модификации
   расширение модуля не ведет к изменениям в исходном коде модуля

Ключом является абстракция

Предположим, к нам пришел заказчик и сказал, что нужно создать приложение-калькулятор, которое может выполнять несколько операций.

В данный момент времени он нам заплатил за сложение и вычитание.

```
public interface CalculatorOperation {

}
```

```
public class Addition implements CalculatorOperation {
    private double firstNumber;
    private double secondNumber;
    private double result = 0.0;

    public Addition(double firstNumber,
                    double secondNumber) {
        this.firstNumber = firstNumber;
        this.secondNumber = secondNumber;
    }

    // getters and setters …
```

```
public class Subtraction implements CalculatorOperation {
    private double firstNumber;
    private double secondNumber;
    private double result = 0.0;

    public Subtraction(double firstNumber,
                       double secondNumber) {
        this.firstNumber = firstNumber;
        this.secondNumber = secondNumber;
    }

    // getters and setters …
```

```
public class Calculator {

    public void calculate(CalculatorOperation operation) {
        if (operation == null) {
            throw new InvalidParameterException("Can not perform operation");
        }

        if (operation instanceof Addition) {
            Addition addition = (Addition) operation;
            addition.setResult(addition.getFirstNumber() + addition.getSecondNumber());
        } else if (operation instanceof Subtraction) {
            Subtraction subtraction = (Subtraction) operation;
            subtraction.setResult(subtraction.getFirstNumber() - subtraction.getSecondNumber());
        }
    }
}
```

Когда заказчик придет к нам второй раз и скажет добавить функцию умножения или деления, у нас нет другого выхода, кроме как изменить метод calculate класса Calculator - **этот код не соответствует OCP**

Код в методе calculate будет меняться с каждым новым запросом на добавление операции.

Поэтому необходимо извлечь этот код и поместить его на уровень абстракции.

```
public interface CalculatorOperation {
    void performOperation();
}
```

```
public class Addition implements CalculatorOperation {
    private double firstNumber;
    private double secondNumber;
    private double result = 0.0;

    public Addition(double firstNumber,
                    double secondNumber) {
        this.firstNumber = firstNumber;
        this.secondNumber = secondNumber;
    }

    @Override
    public void performOperation() {
        result = firstNumber + secondNumber;
    }

     // getters and setters …
```

```
public class Subtraction implements CalculatorOperation {
    private double firstNumber;
    private double secondNumber;
    private double result = 0.0;

    public Subtraction(double firstNumber,
                       double secondNumber) {
        this.firstNumber = firstNumber;
        this.secondNumber = secondNumber;
    }

    @Override
    public void performOperation() {
        result = firstNumber - secondNumber;
    }

    // getters and setters …
```

```
public class Calculator {

    public void calculate(CalculatorOperation operation) {
        if (operation == null) {
            throw new InvalidParameterException("Can not perform operation");
        }

        operation.performOperation();
    }
}
```

```
public class Division implements CalculatorOperation {
    private double fistNumber;
    private double secondNumber;
    private double result;

    @Override
    public void performOperation() {
        if (secondNumber != 0) {
            result = fistNumber / secondNumber;
        } else throw new RuntimeException("Деление на ноль!");
     }

    // getters and setters ...
}
```

IThub

Принцип подстановки Лисков

Liskov Substitution Principle

Подтипы должны быть замещаемыми своими базовыми типами

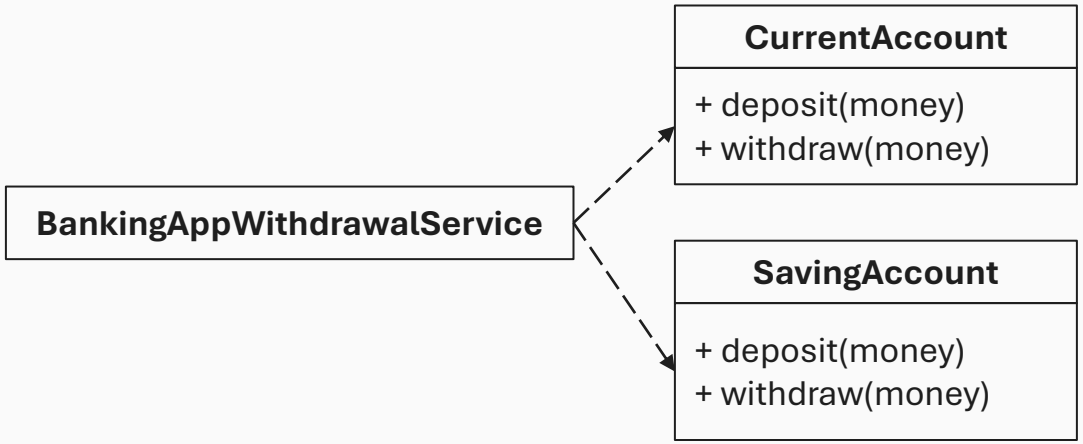
Наследующий класс должен дополнять, а не замещать поведение базового класса.

Например, если есть класс A и унаследованный от него класс B, то если заменить все использования класса A на B, ничего не должно измениться в работе программы (класс B всего лишь расширяет функционал класса A).

Если эта проверка работает, то ваш код соответствует принципу подстановки Лисков

Предположим, к нам пришел заказчик и сказал, что нужно создать банковское приложение. В данный момент он заплатил за разработку логики со сберегательными счетами.

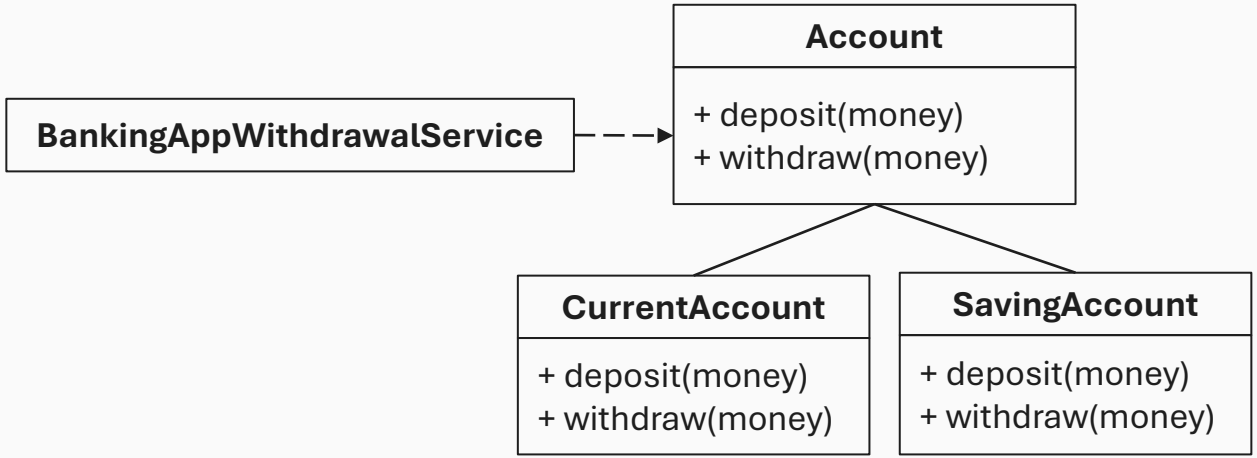
Есть два типа счетов - текущий счет (CurrentAccount) и сберегательный счет (SavingsAccount)



BankingAppWithdrawalService поддерживает две конкретные реализации account.

Поэтому BankingAppWithdrawalService потребуется изменять каждый раз при добавлении нового типа счета.

Чтобы решение соответствовало принципу открытости/закрытости необходимо закрыть BankingAppWithdrawalService от изменений при необходимости добавления новых типов счетов, используя вместо этого базовый класс Account



IThub

```
public abstract class Account {

    protected abstract void withdraw(Long money);
}
```

```
public class BankingAppWithdrawalService {

    private Account account;

    public BankingAppWithdrawalService(Account account) {
        this.account = account;
    }

    public void withdraw(Long money) {
        account.withdraw(money);
    }
}
```

IThub

Теперь банк хочет предложить своим клиентам срочный депозитный счет с высоким процентным доходом.

Для этого введем новый класс FixedTermDepositAccount. Срочный депозитный счет - это тип счета. Это подразумевает наследование.

```
public class FixedTermDepositAccount extends Account {

    @Override
    protected void withdraw(Long money) {

    }
}
```

НО банк не разрешает снятие средств со срочных депозитов

FixedTermDepositAccount не может полноценно предоставлять метод вывода средств, определяемый Account.

Как вариант, можем выбросить исключение UnsupportedOperationException.

```
public class FixedTermDepositAccount extends Account {

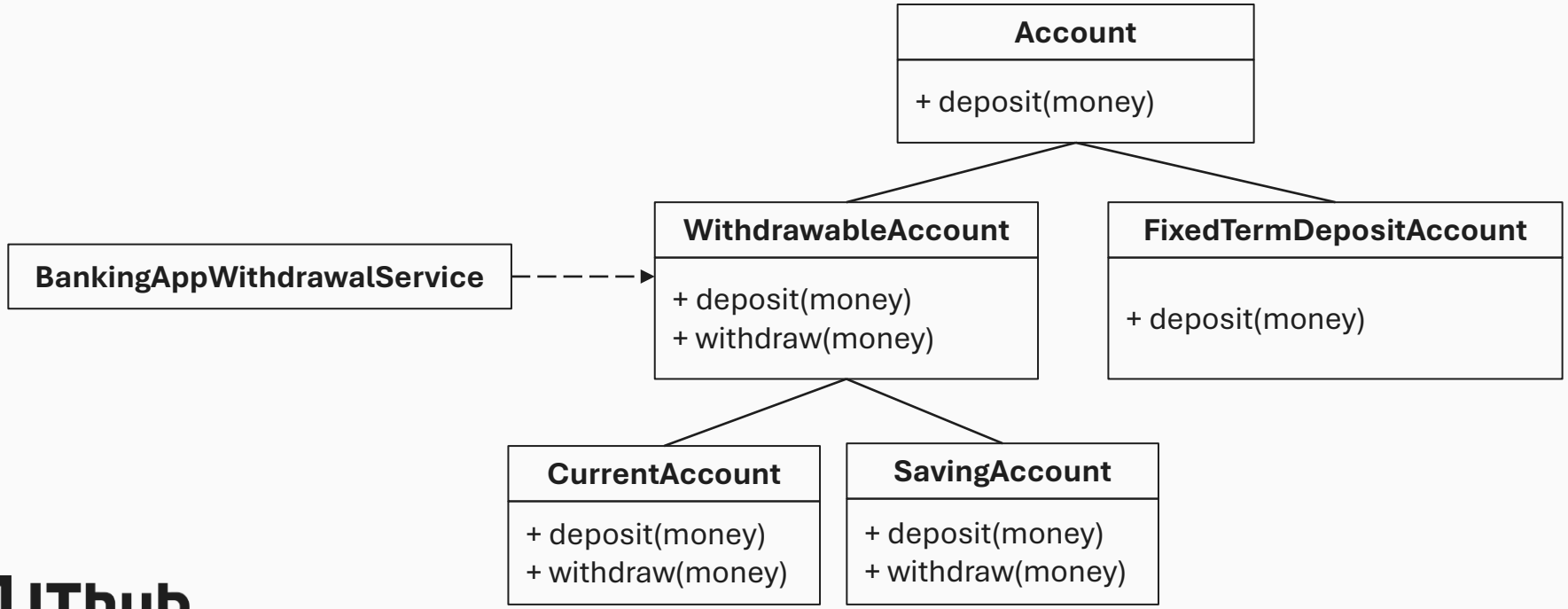
    @Override
    protected void withdraw(Long money) {
        throw new UnsupportedOperationException(“Нельзя выводить деньги!");
    }
}
```

Если мы напишем модульный тест, то приложение будет вылетать с ошибкой.

Можем написать throws в методе withdraw. Это нарушает принцип открытости/закрытости.

Мы не можем надёжно заменить FixedTermDepositAccount на Account → нарушение принципа подстановки Лисков

При проектировании Account **ошибочно** предполагалось, что все типы Account допускают вывод средств.



IThub

IThub

```
public class BankingAppWithdrawalService {

    private WithdrawableAccount withdrawableAccount;

    public BankingAppWithdrawalService(WithdrawableAccount account) {
        this.withdrawableAccount = account;
    }

    public void withdraw(Long money) {
        withdrawableAccount.withdraw(money);
    }
}
```

```
public abstract class Account {

    protected abstract void deposit(Long money);
}
```

```
public class FixedTermDepositAccount
                        extends Account {

    @Override
    protected void deposit(Long money) {
        // some logic
    }
}
```

FixedTermDepositAccount сохраняет наследование от Account и переопределяет метод deposit, который он может надежно реализовать, и больше не наследует метод вывода денег, который ему не нужен.

Генерация исключений в производных классах

Еще одна форма нарушения принципа LSP выглядит как добавление в методы производных классов исключений, которые не генерировались соответствующими методами базовых классов

- Либо ожидания пользователей понадобится изменить (и изменить методы в базовых классах)
- Либо производные классы не должны генерировать исключения

```java
public class Animal {

    private String color;
    private int age;
    private int weight;

    // constructor, getters and setters

    public void eat() {
        System.out.println(
          "This animal eats food.");
    }

    public void sleep() {
        System.out.println(
          "This animal is sleeping zzzz");
    }
}
```

```java
public class Dog extends Animal {

    private String name;
    private String breed;

    // constructor, getters and setters

    @Override
    public void eat() {
        if (this.name != null && !this.name.isEmpty()) {
            System.out.println("The dog " + this.name + " eats meat");
        } else {
            System.out.println("The dog eats meat");
        }
    }

    @Override  new *
    public void sleep() {
        if (this.breed == "Non sleeping dog") {
            System.out.println("this dog doesn't sleep!");
            throw new MyException();
        } else {
            System.out.println
        }
    }
}
```

Unhandled exception: org.example.exceptions.MyException

Add exception to method signature Alt+Shift+Enter More a

© org.example.exceptions.MyException

public MyException()

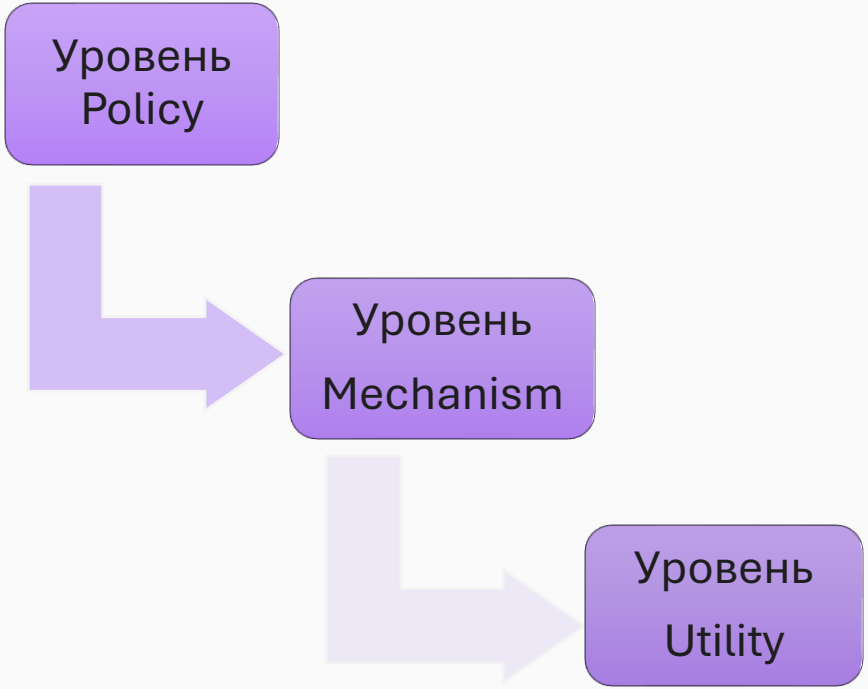
IThub

Принцип инверсии зависимостей

Dependency inversion principle

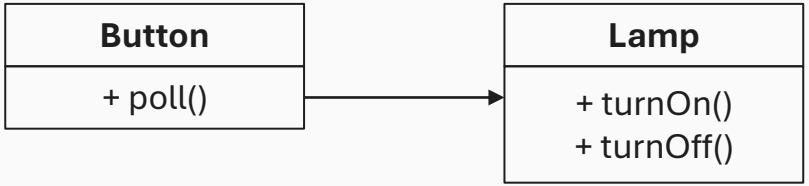
1. Модули верхнего уровня не должны зависеть от модулей нижнего уровня.
Те и другие должны зависеть от абстракций

2. Абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций



- уровень Policy чувствителен к изменениям на всем пути вниз к уровню Utility
- эта зависимость транзитивна

- Объект **Button** (кнопка) распознает, что пользователь либо включил, либо выключил свет, получив сообщение Poll
- **Lamp** (лампочка) оказывает воздействие:
  - сообщение TurnOn – включает свет
  - сообщение TurnOff – выключает свет

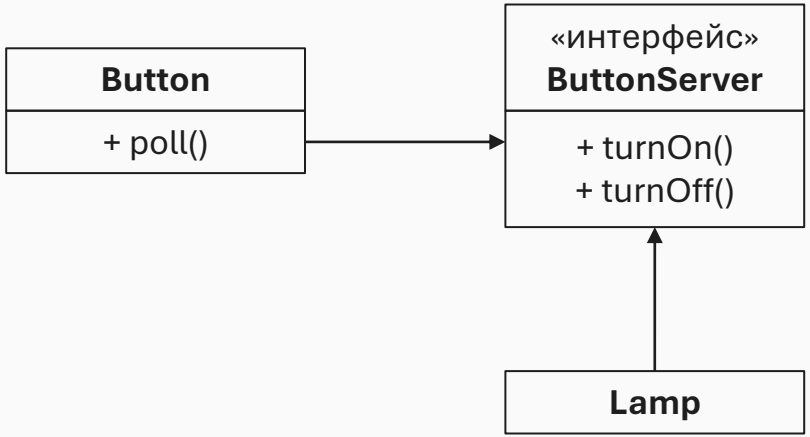


класс Button напрямую зависит от класса Lamp

```
public class Button {
    private Lamp itsLamp;

    public void poll() {
        if (/* некоторое условие*/) {
            itsLamp.turnOn();
        }
    }
}
```

Это решение нарушает принцип инверсии зависимостей!



- Lamp зависит от ButtonServer, но ButtonServer вовсе не зависит от Button
- Любой объект, которому известно как манипулировать интерфейсом ButtonServer, сможет управлять объектом Lamp

IThub

Порождающие шаблоны



отделить процесс создания объектов от кода

скрывают сложную логику создания объектов

полезны в больших проектах, где объекты имеют сложную структуру

упрощают добавление новых реализаций

Порождающие шаблоны проектирования (Creational Design Patterns) — это группа паттернов, которые отвечают за создание объектов в программном обеспечении.

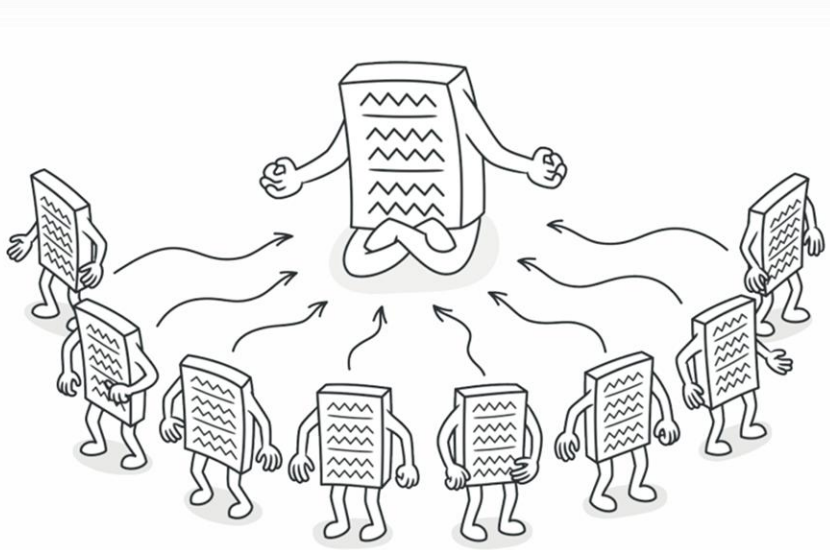
Основные порождающие паттерны

- Singleton (Одиночка)
- Builder (Строитель)
- Prototype (Прототип)
- Abstract Factory (Абстрактная фабрика)
- Factory Method (Фабричный метод)

Singleton
одиночка

IThub

Singleton — это паттерн, который ограничивает создание экземпляров класса одним объектом в рамках определённого контекста и предоставляет общую точку доступа к нему.



Его часто реализуют через статичное поле:

IThub

```
public class GameConfig {
    private static final GameConfig INSTANCE = null;

    private GameConfig() {
    }

    public static GameConfig getInstance() {
        if (INSTANCE == null) {
            INSTANCE = new GameConfig();
        }
        return INSTANCE;
    }
}
```

В этом примере конструктор приватный, поэтому создать объект с помощью new GameConfig() извне нельзя.
Метод getInstance() возвращает единственный экземпляр, созданный этим классом.

```
@Getter
@Setter
static class GameSettings {
        private static final GameSettings INSTANCE = null;

        private int difficulty = 2;
        private int masterVolume = 80;

        private GameSettings() {}

        public static GameSettings getInstance() {
            if (INSTANCE == null) {
              INSTANCE = new GameSettings();
            }
            return INSTANCE;
        }
}
```

IThub

Где-то в коде:

```
GameSettings settings = GameSettings.getInstance();
settings.setDifficulty(3);

….

if(settings.getInstance().getDifficulty() > 3) {
      // some logic
}
```

Класс не должен быть пустым, можно добавлять поля, например:
difficulty – уровень сложности
masterVolume – общая громкость звука

Singleton
одиночка

IThub

✅ централизованный доступ к общему объекту

✅ контролируемое создание экземпляра

✅ простая реализация в базовом варианте

✅ если Singleton не используется, то он и не создается

❌ глобальное состояние усложняет тестирование

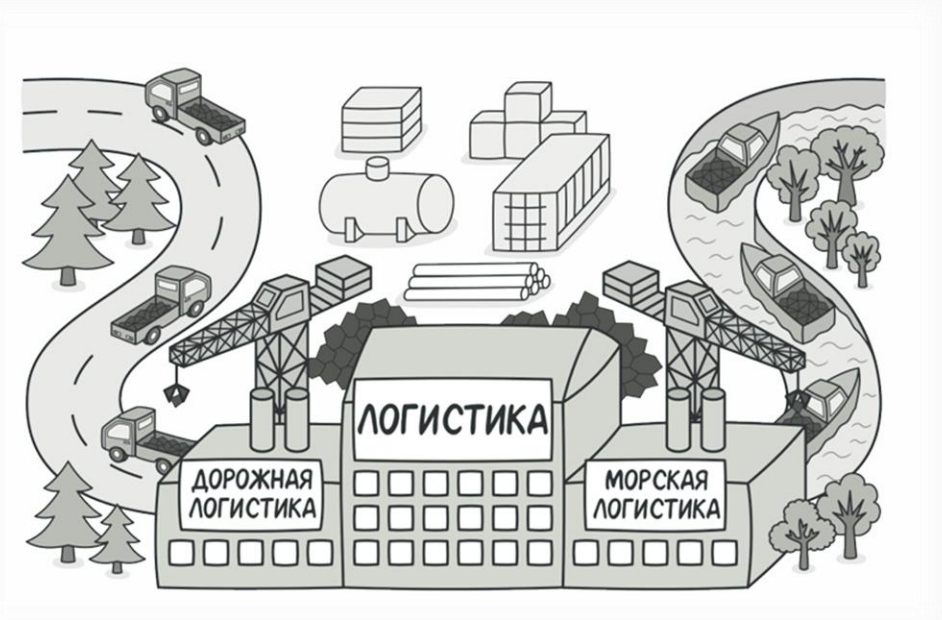
❌ при неосторожном использовании возникают проблемы с многопоточностью и жизненным циклом объекта

❌ не существует хорошего способа уничтожения Singleton

Factory Method
фабричный метод

IThub

Factory Method — это паттерн, который определяет метод создания объекта, оставляя подклассам возможность выбирать конкретный тип создаваемого продукта.



Пример

IThub

Представим, что у нас программа управления грузовыми перевозками.

Сперва мы рассчитываем перевозить товары только на автомобилях. Поэтому весь ваш код работает с объектами класса **Грузовик**.

В какой-то момент наша программа становится настолько известной, что морские перевозчики выстраиваются в очередь и просят добавить поддержку морской логистики в программу.



Добавить новый класс не так-то просто, если весь код уже завязан на конкретные классы.

IThub

Большая часть существующего кода жёстко привязана к классам **Грузовиков**.

Чтобы добавить в программу классы **морскихСудов**, понадобится перелопатить всю программу.

Более того, если мы потом решим добавить в программу ещё один вид транспорта, то всю эту работу придётся повторить.

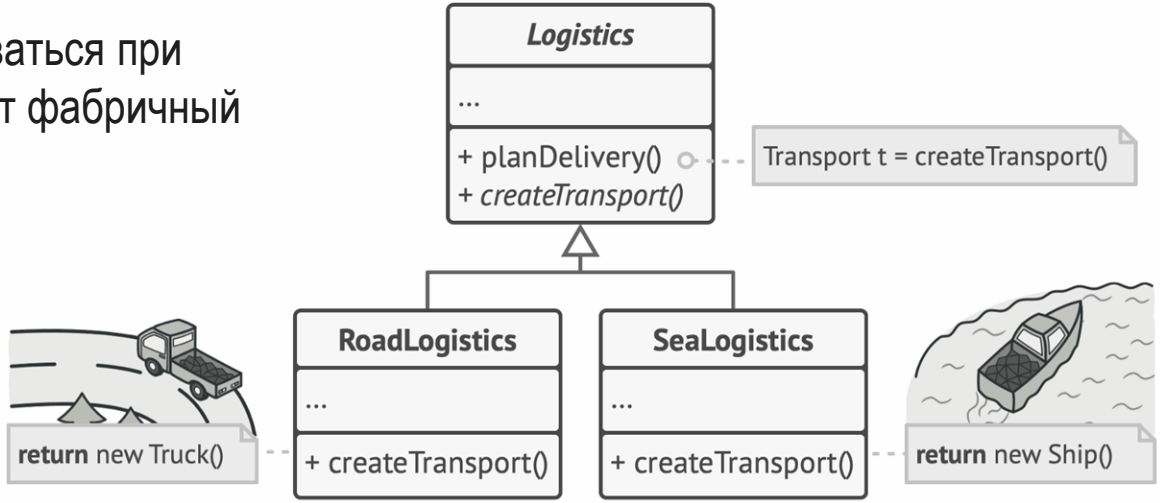
В итоге мы получим ужасающий код, наполненный условными операторами, которые выполняют то или иное действие, в зависимости от класса транспорта.

Решение

IThub

Паттерн **Фабричный метод** предлагает создавать объекты не напрямую, используя оператор new, а через вызов особого фабричного метода.

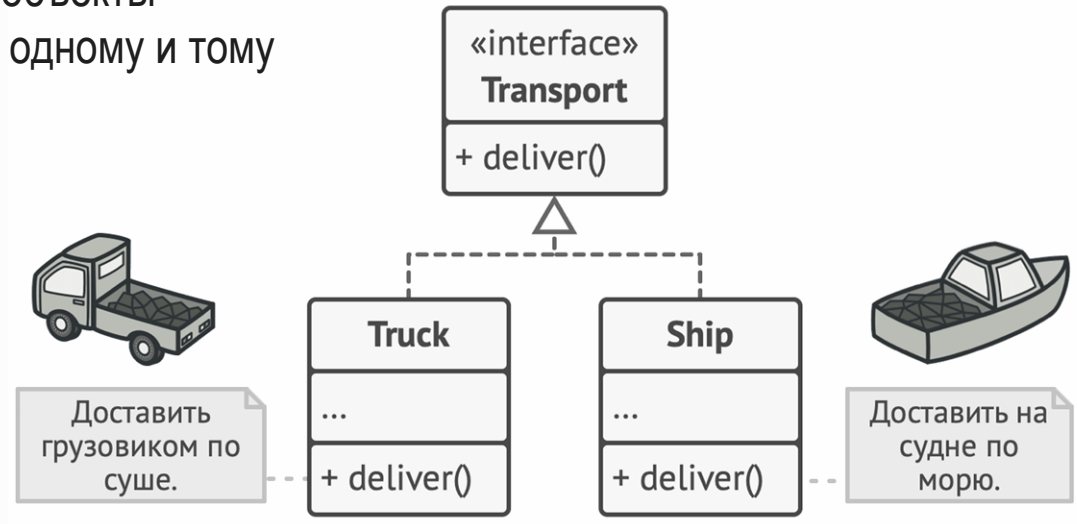
Объекты всё равно будут создаваться при помощи new, но делать это будет фабричный метод.



Подклассы могут изменять класс создаваемых объектов.

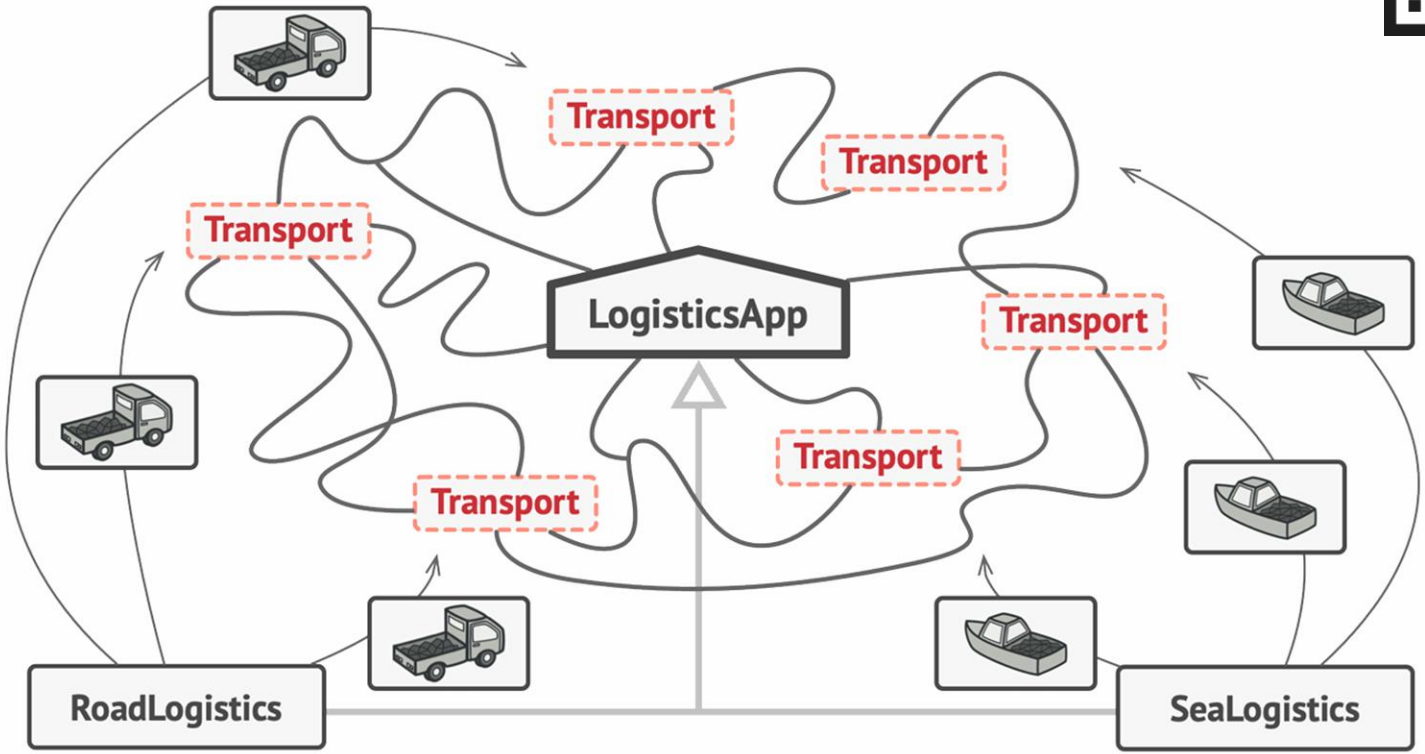
Чтобы эта система заработала, все возвращаемые объекты должны иметь общий интерфейс.

Подклассы смогут производить объекты различных классов, следующих одному и тому же интерфейсу.

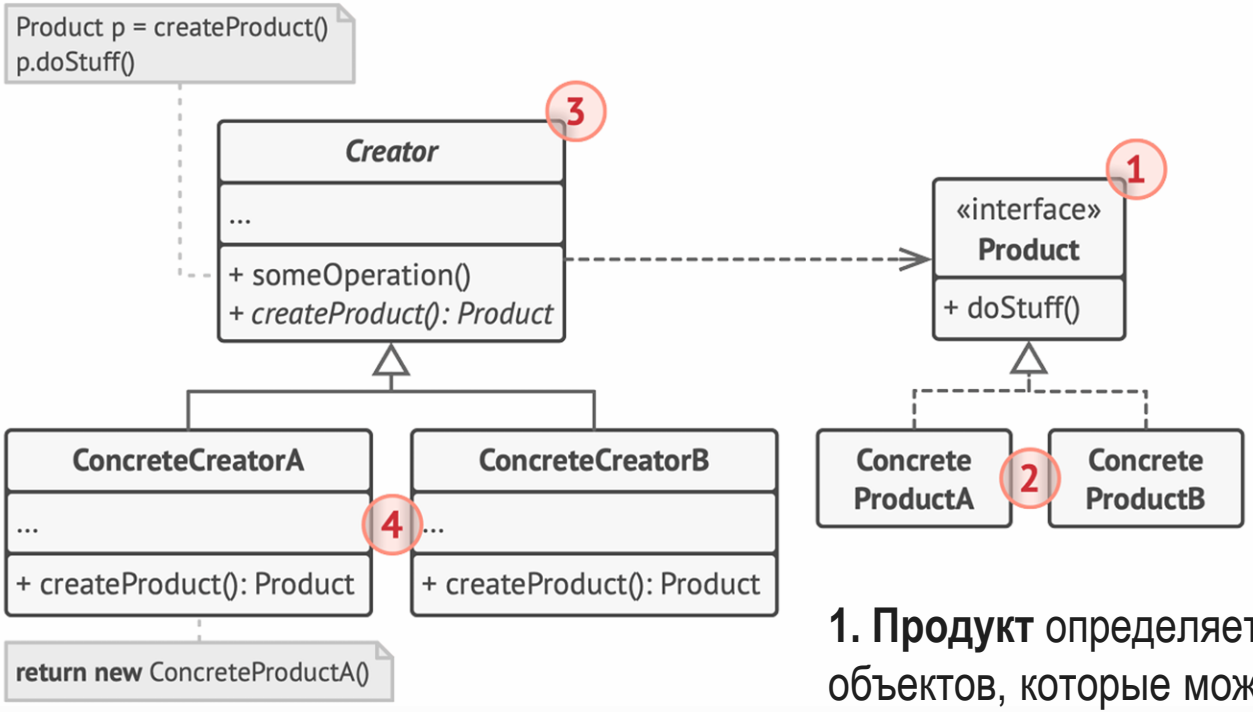


Все объекты-продукты должны иметь общий интерфейс.

IThub

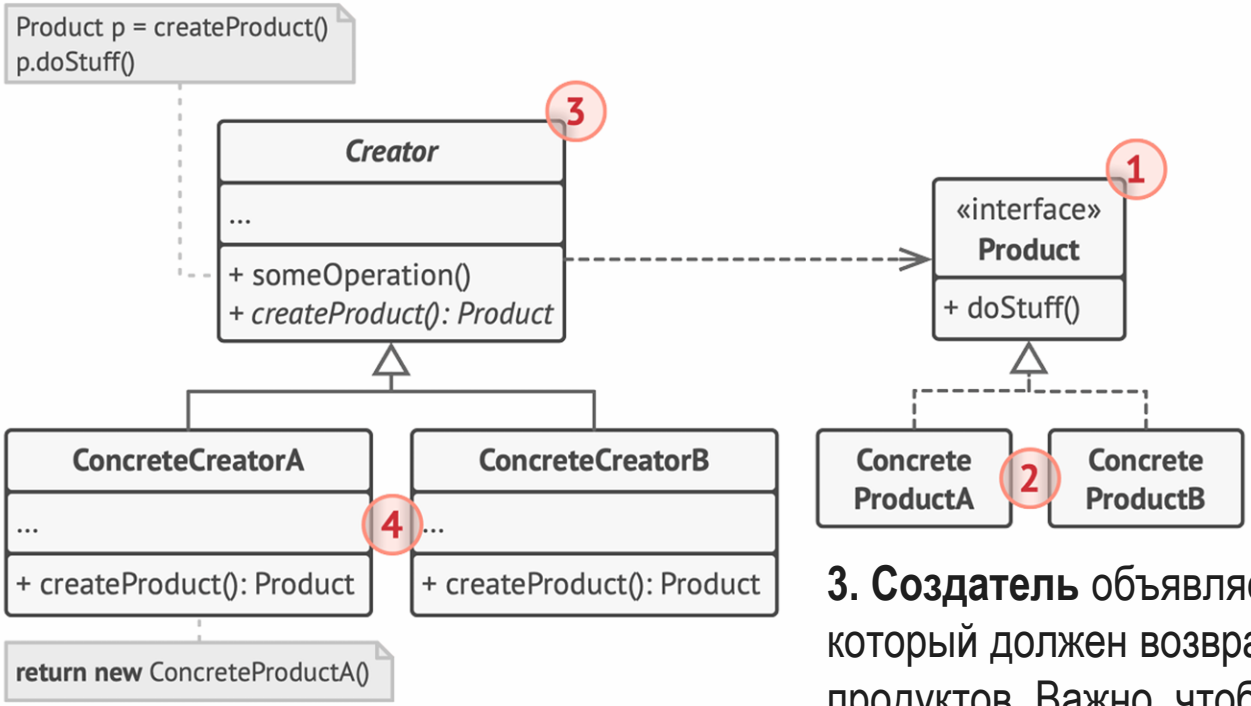


Пока все продукты реализуют общий интерфейс, их объекты можно взаимозаменять в клиентском коде.



1. Продукт определяет общий интерфейс объектов, которые может произвести создатель и его подклассы.

2. Конкретные продукты содержат код различных продуктов. Продукты будут отличаться реализацией, но интерфейс у них будет общий.



3. Создатель объявляет фабричный метод, который должен возвращать новые объекты продуктов. Важно, чтобы тип результата совпадал с общим интерфейсом продуктов.

4. Конкретные создатели по-своему реализуют фабричный метод, производя те или иные конкретные продукты.

Применение паттерна в RPG игре

```
abstract class Monster {
    public abstract void attack();
}
```

```
class Goblin extends Monster {
    @Override
    public void attack() {
        System.out.println("Гоблин атакует кинжалом!");
    }
}
```

Monster задаёт общий тип противника, а конкретные классы определяют поведение каждого монстра.

```
class Skeleton extends Monster {
    @Override
    public void attack() {
        System.out.println("Скелет атакует ржавым мечом!");
    }
}
```

IThub

IThub

```
abstract class MonsterSpawner {

    // Factory Method
    protected abstract Monster createMonster();

    // общая логика для всех создателей
    public void startBattle() {
        Monster monster = createMonster();

        System.out.println("На героя нападает монстр!");
        monster.attack();
    }
}
```

```
class ForestSpawner extends MonsterSpawner {
    @Override
    protected Monster createMonster() {
        return new Goblin();
    }
}
```

```
class DungeonSpawner extends MonsterSpawner {
    @Override
    protected Monster createMonster() {
        return new Skeleton();
    }
}
```

Где-то в коде:

```
public static void main(String[] args) {
        MonsterSpawner spawner = new ForestSpawner();
        spawner.startBattle();

        System.out.println("--- Подземелье ---");

        spawner = new DungeonSpawner();
        spawner.startBattle();
    }
```

```
На героя нападает монстр!
Гоблин атакует кинжалом!
--- Подземелье ---
На героя нападает монстр!
Скелет атакует ржавым мечом!
```

Factory Method
фабричный метод

IThub

✅ уменьшает зависимость от конкретных классов

✅ упрощает добавление новых типов продуктов

✅ позволяет делегировать создание объектов специализированным подклассам

❌ может потребовать создания большого количества классов

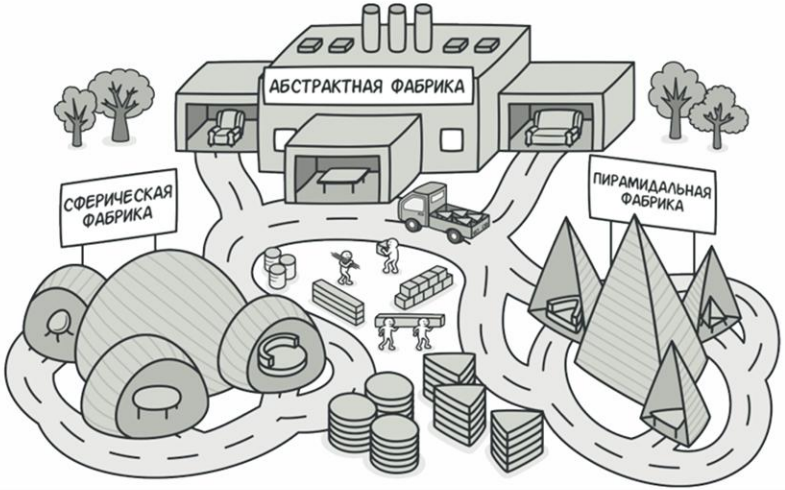
❌ структура проекта становится сложнее, если типов продуктов мало

Abstract Factory
абстрактная фабрика

IThub

Abstract Factory — это паттерн для создания семейств связанных объектов без указания их конкретных классов.

Он полезен, когда программе необходимо работать с несколькими совместимыми наборами компонентов.



Пример

IThub

Представим, что мы пишем систему для мебельного магазина.

У нас есть семейство зависимых продуктов **Диван** + **Столик** + **Кресло.**

И есть несколько вариаций этого семейства. Например, продукты **Кресло**, **Диван** и **Столик** представлены в трёх разных стилях: Классическом, Викторианском и Модерне.

Нам нужен такой способ создавать объекты продуктов, чтобы они сочетались с другими продуктами того же семейства.

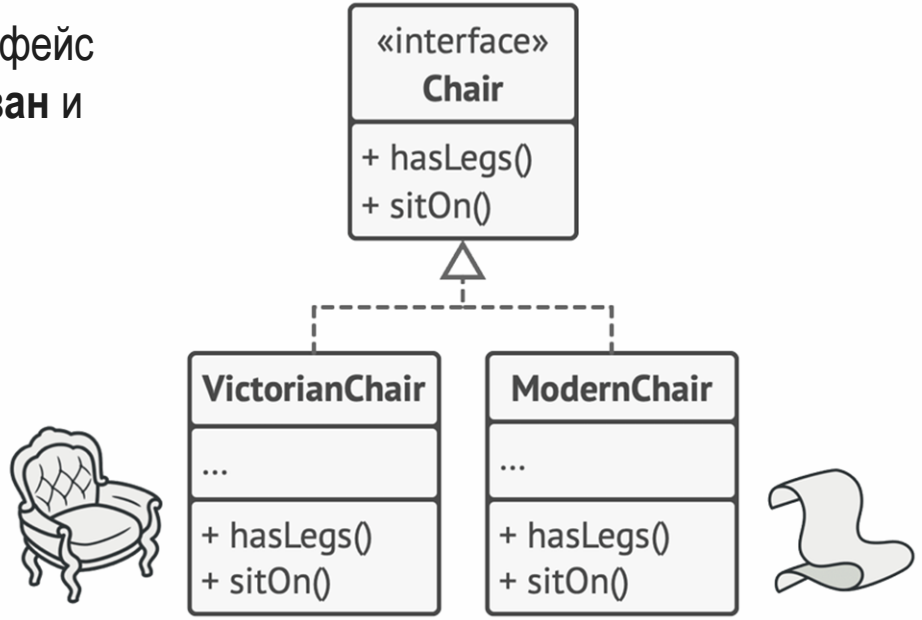


Клиенты расстраиваются, если получают несочетающиеся продукты.

IThub

Паттерн Абстрактная фабрика предлагает выделить общие интерфейсы для отдельных продуктов, составляющих семейства.

Так, все вариации кресел получат общий интерфейс **Кресло**, все диваны реализуют интерфейс **Диван** и так далее.



Все вариации одного и того же объекта должны жить в одной иерархии классов.

IThub

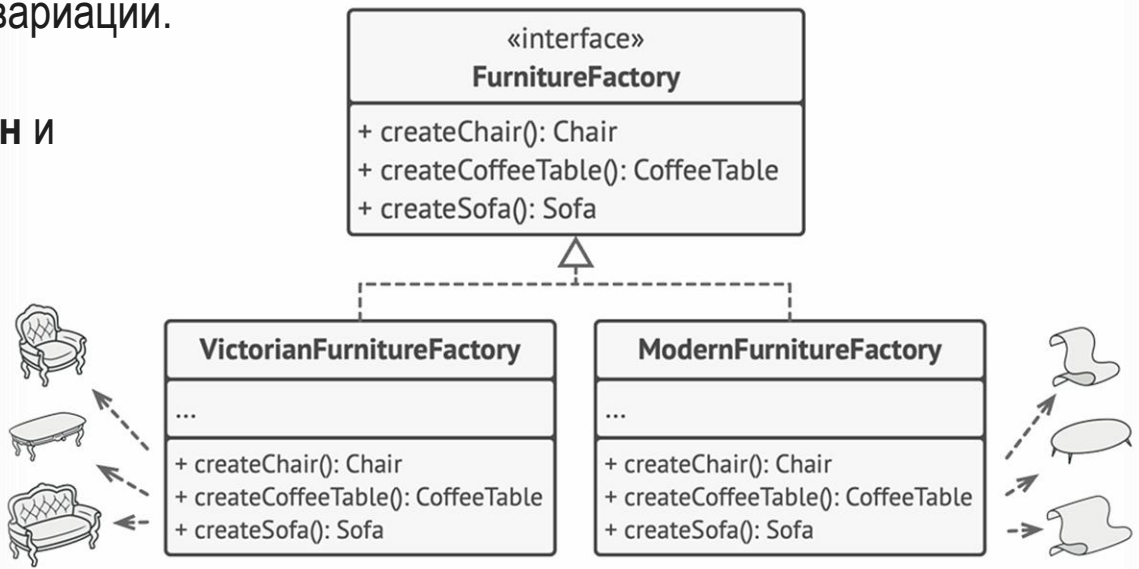
Далее мы создаём абстрактную фабрику — общий интерфейс, который содержит методы создания всех продуктов семейства (например, **создатьКресло**, **создатьДиван** и **создатьСтолик**).

Эти операции должны возвращать абстрактные типы продуктов, представленные интерфейсами, которые выделили ранее - **Кресла**, **Диваны** и **Столики**.

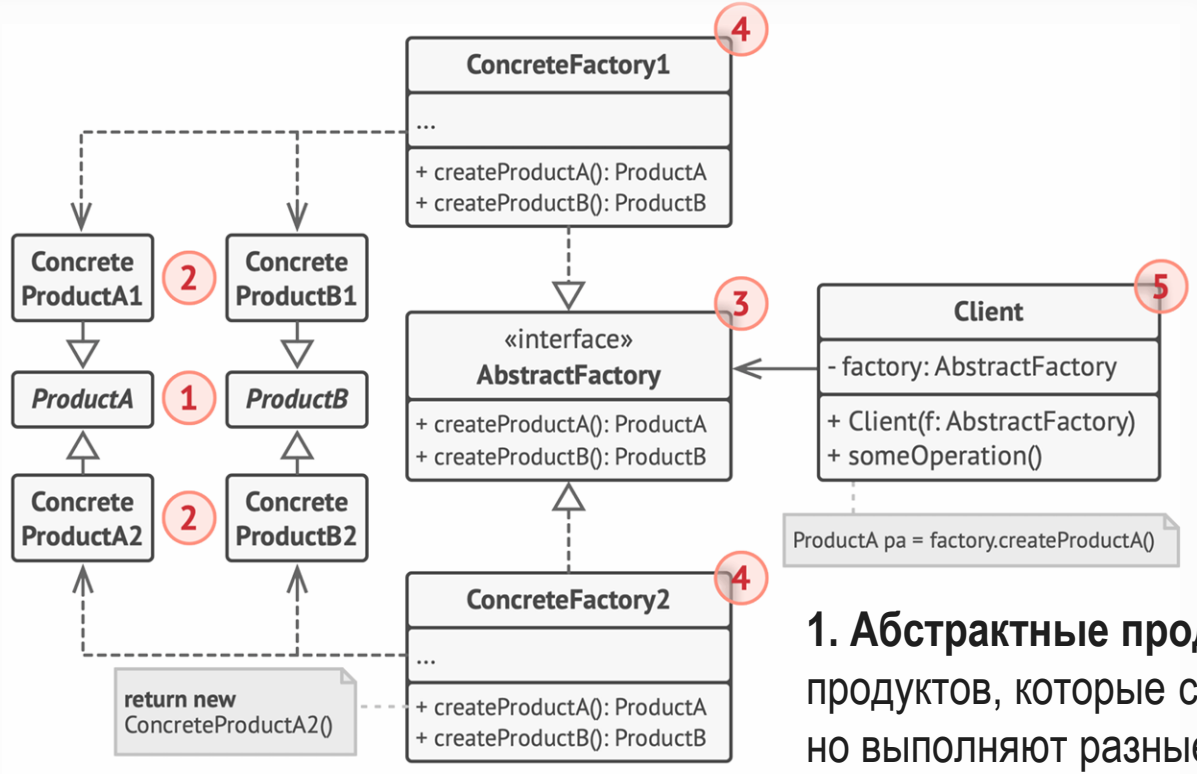
IThub

Для каждой вариации семейства продуктов мы должны создать свою собственную фабрику, реализовав абстрактный интерфейс.

Фабрики создают продукты одной вариации. Например, **ФабрикаМодерн** будет возвращать только **ДиваныМодерн** и **СтоликиМодерн**.



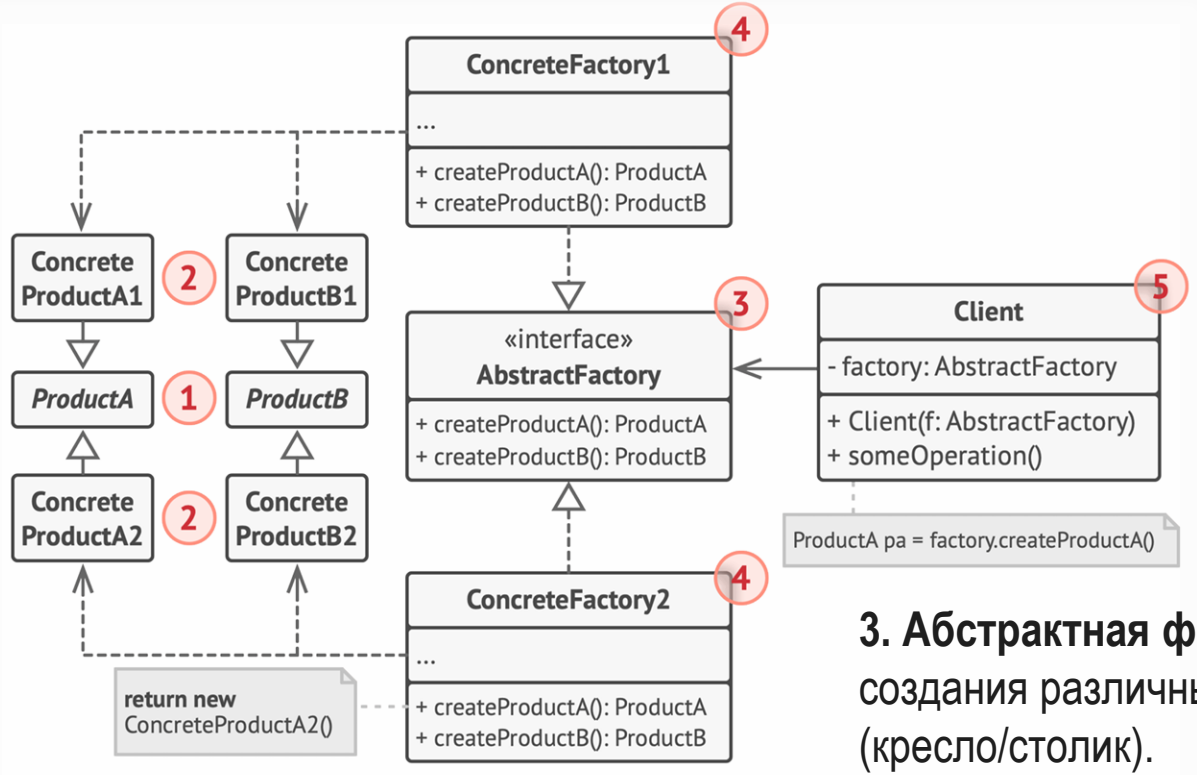
Конкретные фабрики соответствуют определённой вариации семейства продуктов.



IThub

1. Абстрактные продукты объявляют интерфейсы продуктов, которые связаны друг с другом по смыслу, но выполняют разные функции

2. Конкретные продукты большой набор классов, которые относятся к различным абстрактным продуктам (кресло/столик), но имеют одни и те же вариации (Викторианский/Модерн)



3. Абстрактная фабрика объявляет методы создания различных абстрактных продуктов (кресло/столик).

4. Конкретные фабрики относятся каждая к своей вариации продуктов (Викторианский/Модерн) и реализуют методы абстрактной фабрики, позволяя создавать все продукты определённой вариации.

Применение паттерна в RPG игре

```
// оружие
interface Weapon {
        void attack();
}
```

```
// броня
interface Armor {
        void defend();
}
```

```
class FireSword implements Weapon {
        @Override
        public void attack() {
            System.out.println("Огненный меч наносит урон огнём!");
        }
    }
```

```
class IceSword implements Weapon {
        @Override
        public void attack() {
            System.out.println("Ледяной меч замораживает врага!");
        }
    }
```

```
class FireArmor implements Armor {
        @Override
        public void defend() {
            System.out.println("Огненная броня защищает от жара!");
        }
    }
```

```
class IceArmor implements Armor {
        @Override
        public void defend() {
            System.out.println("Ледяная броня отражает холод!");
        }
    }
```

IThub

IThub

```
interface EquipmentFactory {
    Weapon createWeapon();
    Armor createArmor();
}
```

```
class FireEquipmentFactory implements EquipmentFactory {
        public Weapon createWeapon() {
            return new FireSword();
        }

        public Armor createArmor() {
            return new FireArmor();
        }
    }
```

```
class IceEquipmentFactory implements EquipmentFactory {
        public Weapon createWeapon() {
            return new IceSword();
        }

        public Armor createArmor() {
            return new IceArmor();
        }
    }
```

где-то в коде:

```
EquipmentFactory factory = new FireEquipmentFactory();

Weapon weapon = factory.createWeapon();
Armor armor = factory.createArmor();
```

Abstract Factory
абстрактная фабрика

IThub

✅ гарантирует согласованность объектов одного семейства

✅ позволяет заменять целые наборы компонентов

✅ скрывает детали создания объектов

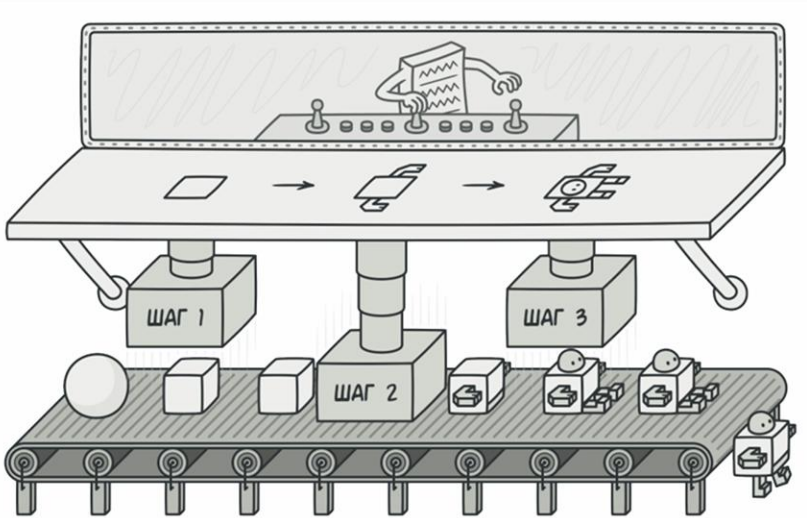
❌ количество интерфейсов и классов увеличивается

Builder
строитель

IThub

Builder — это паттерн, который позволяет создавать сложный объект пошагово, отделяя процесс его сборки от представления результата.

Он особенно полезен, когда объект имеет много параметров, часть из которых необязательна.



Пример

IThub

Настройки игрового персонажа в игре могут включать имя, здоровье, скорость, броню, набор способностей и еще кучу параметров.

Код инициализации таких объектов обычно спрятан внутри монструозного конструктора с десятком параметров.

Либо ещё хуже —распылён по всему клиентскому коду.

Паттерн **Строитель** предлагает вынести конструирование объекта за пределы его собственного класса, поручив это дело отдельным объектам, называемым **строителями.**

```
class Character {
    String name;
    int health;

    Character(Builder builder) {
        this.name = builder.name;
        this.health = builder.health;
}

    static class Builder {
        private String name = "Герой";
        private int health = 100;

        Builder name(String name) {
            this.name = name;
            return this;
        }

        Builder health(int health) {
            this.health = health;
            return this;
        }

        Character build() {
            return new Character(this);
        }
    }
}
```

Паттерн предлагает разбить процесс конструирования объекта на отдельные шаги:

```
Character hero = new Character.Builder()
        .name("Mar")
        .health(150)
        .build();
```

Builder
строитель

IThub

✅ повышает читаемость создания сложных объектов

✅ позволяет задавать необязательные параметры

✅ скрывает детали создания объектов

❌ клиент будет привязан к конкретным классам строителей, так как в интерфейсе директора может не быть метода получения результата

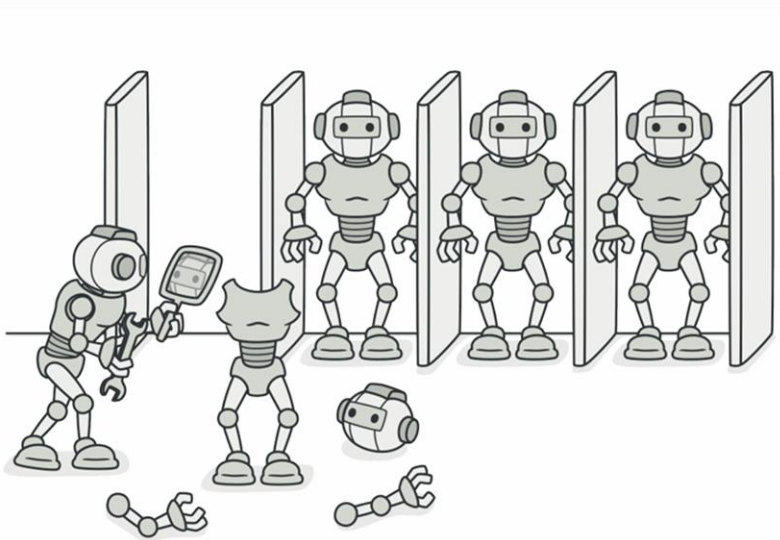
❌ усложняет код программы из-за введения дополнительных классов

Prototype
прототип

IThub

Prototype — это паттерн, который позволяет создавать новые объекты на основе копирования существующих экземпляров.

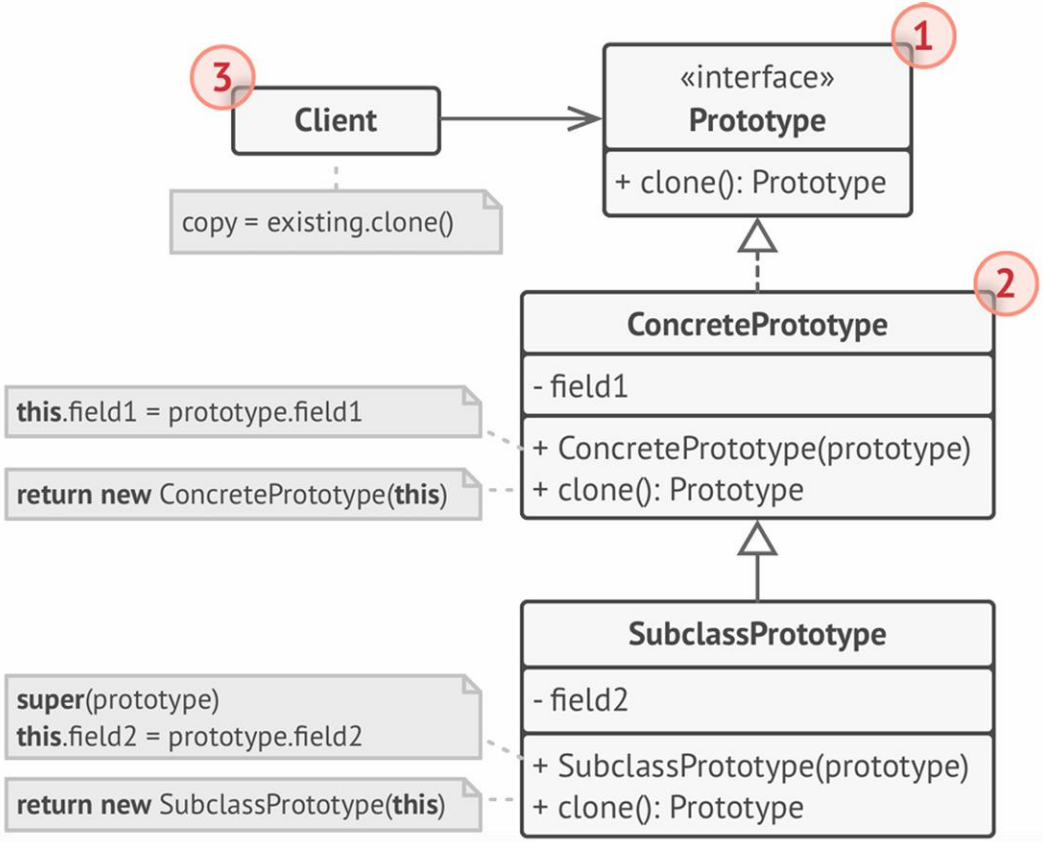
Он полезен, когда создание объекта обходится дорого или требуется быстро получать множество объектов с похожими начальными настройками.



Паттерн **Прототип** поручает создание копий самим копируемым объектам. Он вводит общий интерфейс для всех объектов, поддерживающих клонирование. Это позволяет копировать объекты, не привязываясь к их конкретным классам. Обычно такой интерфейс имеет всего один метод **clone**.

Реализация этого метода в разных классах очень схожа. Метод создаёт новый объект текущего класса и копирует в него значения всех полей собственного объекта. Так получится скопировать даже приватные поля, так как большинство языков программирования разрешает доступ кприватным полям любого объекта текущего класса.

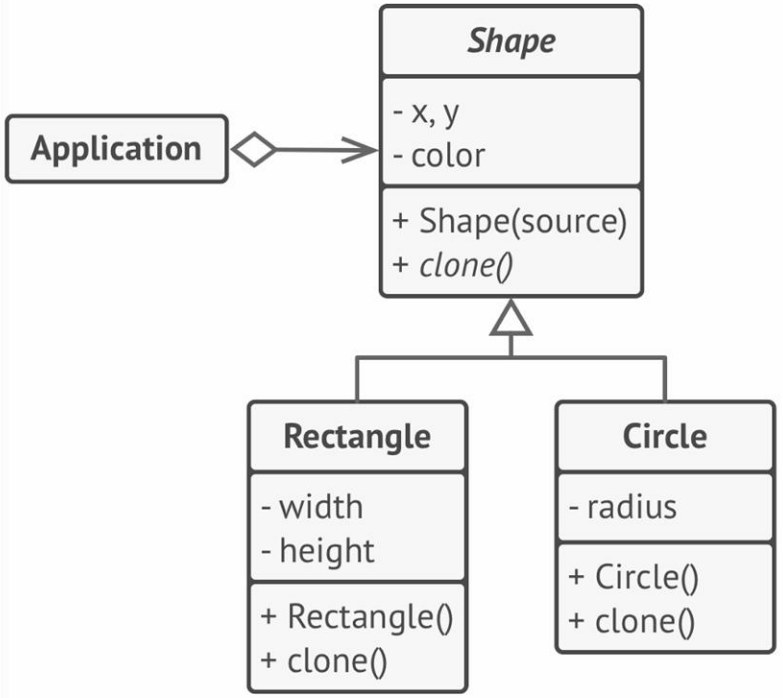
Базовая реализация



1. **Интерфейс прототипов** описывает операции клонирования (обычно метод **clone**)

2. **Конкретный прототип** реализует операцию клонированиясамого себя. Помимо банального копирования значений всех полей, здесь могут быть спрятаны различные сложности, о которых не нужно знать клиенту.

3. **Клиент** создаёт копию объекта, обращаясь к нему через общий интерфейс прототипов.



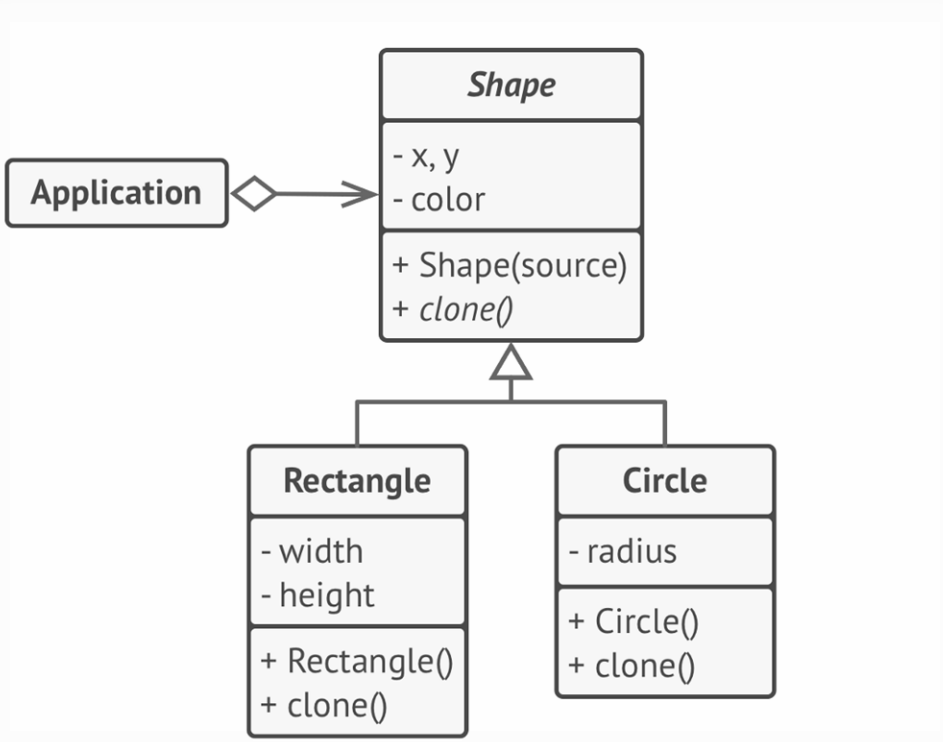
Пример клонирования иерархии геометрических фигур.

```
// базовый прототип
class Shape is
    int x
    int y
    String color

public Shape() {}

public Shape(int x, int y, String color) {
    this.x = x
    this.y = y
    this.color = color
}

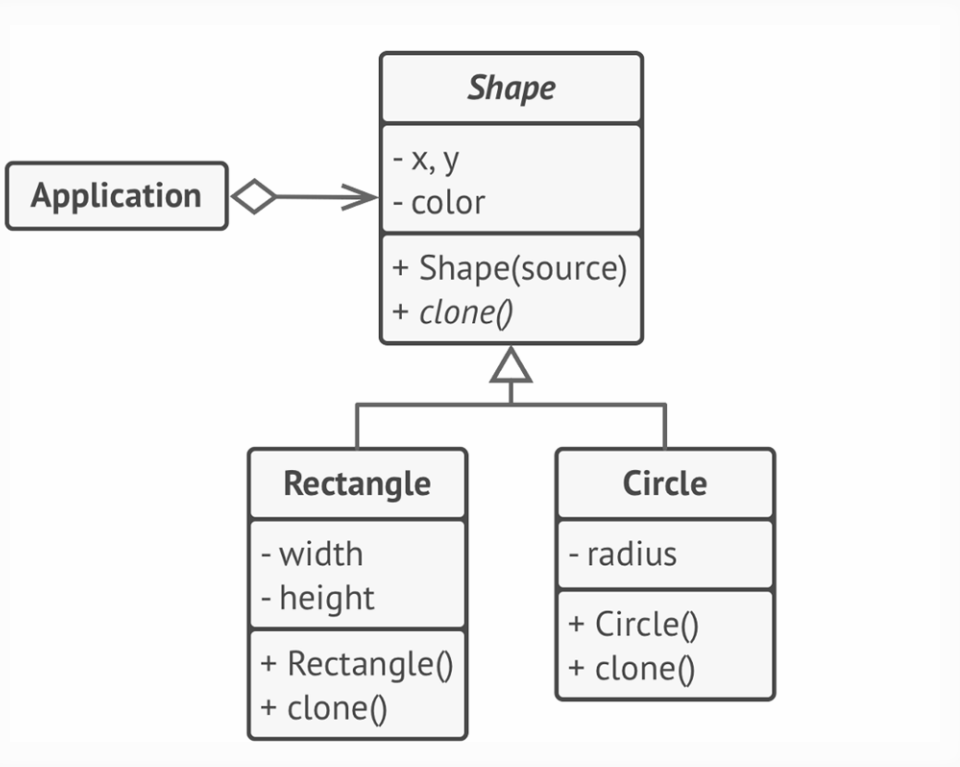
// клонирование
public abstract Shape clone()
```




```
class Rectangle extends Shape
    int width
    int height

public Rectangle(int width, int height) {
    super();
    this.width = width;
    this.height = height;
}

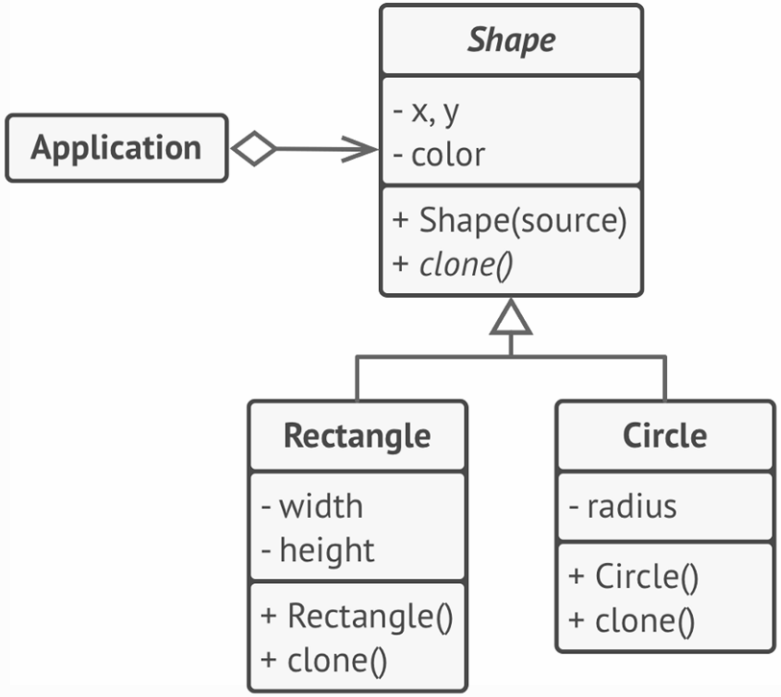
public Shape clone() {
    return new Rectangle(this.width, this.height)
}
```




```
class Circle extends Shape
   int radius

public Circle(int radius) {
    super(source);
    this.radius = radius;
}

public Shape clone() {
    return new Circle(this.radius)
}
```



Где-то в коде:

```
List<Shape> shapes = new ArrayList<>();

Circle circle = new Circle(10);
shapes.add(circle);
Circle anotherCircle = circle.clone();
shapes.add(anotherCircle);

Rectangle rectangle = new Rectangle(10, 20);
shapes.add(rectangle);


// благодаря полиморфизму, мы можем
// клонировать все объекты «вслепую»
// будет выполнен метод clone того класса,
// которым является этот объект
List<Shape> shapesCopy = new ArrayList<>();
for(Shape shape: shapes) {
    shapesCopy.add(shape.clone());
}
```

```
class Enemy implements Prototype<Enemy> {
    private String type;
    private int health;
    private int damage;

    public Enemy(String type, int health, int damage) {
        this.type = type;
        this.health = health;
        this.damage = damage;
    }

    @Override
    public Enemy copy() {
        // создаём отдельный объект с теми же характеристиками
        return new Enemy(type, health, damage);
    }

    public void takeDamage(int damage) {
        health = Math.max(0, health - damage);
    }

    public void showInfo() {
        System.out.println(
            type + " | HP: " + health
            + " | Урон: " + damage
        );
    }
}
```

```
interface Prototype<T> {
    T copy();
}
```

Применение паттерна в RPG игре

где-то в коде:

```
Enemy original = new Enemy("Скелет", 60, 12);
Enemy clone = original.copy();

clone.takeDamage(20);
```

IThub

Prototype
прототип

IThub

✅ ускоряет создание объектов с похожими параметрами

✅ позволяет копировать заранее настроенные экземпляры

✅ снижает зависимость от конкретных классов при правильной организации интерфейса копирования

❌ сложно корректно копировать объекты с вложенной структурой

❌ требуется определить правила копирования состояния

Задание (КТ №7)

IThub

Создать проект, в котором реализован и практически применен **КАЖДЫЙ** из порождающих паттернов проектирования:

1. Singleton
2. FactoryMethod
3. AbstractFactory
4. Builder
5. Prototype

! код должен быть покрыт unit-тестами (junit)

! проект размещается на GitLab\GitHub с настроенным pipeline на тесты

Возможные темы проекта

- игра
- интернет-магазин
- система управления библиотекой
- система управления транспортом
- система бронирования
- система создания и настройки уведомлений

1. RPG

IThub

- создание персонажей разных классов: воин, маг, лучник и т.п.
- настройка имени, здоровья, силы, брони и способностей
- формирование комплектов снаряжения разных фракций
- создание противников на основе готовых шаблонов
- управление общими настройками игры

Паттерн	Реализация
Singleton	Общие настройки игры или реестр конфигурации
Factory Method	Создание конкретного типа персонажа
Abstract Factory	Создание совместимых комплектов снаряжения разных фракций
Builder	Пошаговая сборка персонажа с необязательными характеристиками
Prototype	Копирование заранее настроенных противников

2. Интернет-магазин

IThub

- создание товаров разных категорий: электроника, одежда, книги и т.п.
- формирование заказов из нескольких товаров
- выбор способа доставки
- создание комплектов товаров
- копирование типовых заказов
- хранение общих настроек магазина

Паттерн	**Реализация**
Singleton	Общая конфигурация магазина
Factory Method	Создание товаров разных типов
Abstract Factory	Создание согласованных комплектов товаров для определённой категории или сценария покупки
Builder	Формирование заказа с товарами, адресом доставки, скидкой и способом оплаты
Prototype	Копирование шаблона заказа или типового набора товаров

3. Система управления библиотекой

IThub

- добавление разных типов изданий
- создание карточек книг с названием, автором, годом выпуска и дополнительными параметрами
- формирование наборов изданий по тематике
- копирование карточек для создания похожих записей
- хранение общих настроек каталога

Паттерн	**Реализация**
Singleton	Настройки каталога или единый координатор приложения
Factory Method	Создание книги, журнала или электронного издания
Abstract Factory	Создание комплектов библиотечных материалов для разных типов каталогов или учебных программ
Builder	Сборка карточки издания с необязательными полями
Prototype	Копирование карточки издания для создания похожей записи

4. Система бронирования гостиницы

IThub

- создание номеров разных категорий
- выбор дат заезда и выезда
- указание количества гостей
- добавление завтрака, парковки и других услуг
- создание типовых пакетов проживания
- копирование бронирования для повторного оформления

Паттерн	**Реализация**
Singleton	Общая конфигурация гостиницы
Factory Method	Создание бронирования определённого типа
Abstract Factory	Создание комплектов услуг для разных категорий проживания
Builder	Формирование бронирования с датами, гостями, номером и дополнительными услугами
Prototype	Копирование шаблона бронирования

5. Система создания уведомлений

IThub

- создание электронных писем, SMS и push-уведомлений
- настройка адресата, темы и текста сообщения
- выбор набора уведомлений для разных сценариев
- формирование сложных сообщений с дополнительными параметрами
- копирование шаблонов сообщений

Паттерн	**Реализация**
Singleton	Общие настройки системы уведомлений
Factory Method	Создание уведомления конкретного типа
Abstract Factory	Создание согласованных компонентов доставки для разных каналов или провайдеров
Builder	Сборка сообщения с темой, текстом, адресатом и дополнительными параметрами
Prototype	Копирование шаблона уведомления